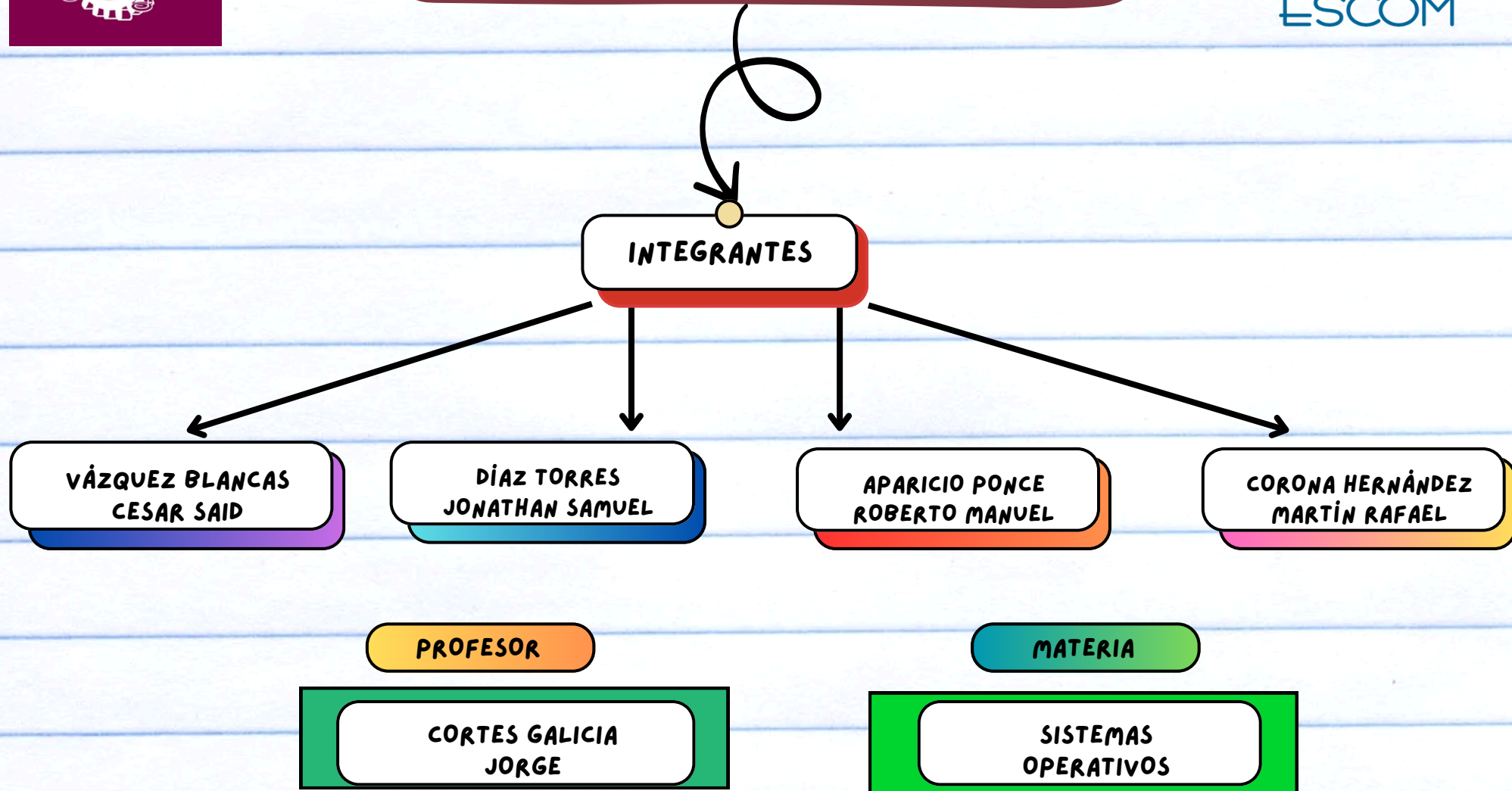


INSTITUTO POLITECNICO NACIONAL ESCUELA SUPERIOR DE COMPUTO



"SOLUCIÓN DE SINCRONIZACIÓN PARA EXCLUSIÓN MUTUA ENTRE DOS PROCESOS"

Descripción de la solución tradicional implementada en software. Restricción para que dos procesos alternen en la ejecución de sus secciones críticas y otras. Uso de las variables "TURN" y "FLAG" para lograr la sincronización. Demostración de la corrección de esta solución en términos de exclusión mutua, progreso y espera finita.

"INTERACCIÓN Y COOPERACIÓN ENTRE PROCESOS EN EL SISTEMA"

Pueden compartir un espacio de direcciones lógico de forma directa o mediante archivos o mensajes. Pueden influir o ser influenciados por otros procesos en el sistema. Procesos cooperativos.

DESARROLLO DE UN MODELO DE SISTEMA CON PROCESOS COOPERATIVOS Y GESTIÓN DE BÚFER LIMITADO

Desarrollo de un modelo de sistema con procesos o hebras secuenciales cooperativas.

Solución inicial con un búfer limitado. Problema con el búfer limitado. Propuesta de solución con variable entera "counter" para mantener el número correcto de elementos en el búfer. Deficiencia en la solución: máximo $\text{búfer.size} - 1$ elementos. Condición de carrera. Importancia de sincronizar los procesos para evitar condiciones de carrera.

SINCRONIZACIÓN DE PROCESOS

GESTIÓN DE LA SECCIÓN CRÍTICA Y PREVENCIÓN DE CONDICIONES DE CARRERA EN SISTEMAS DE KERNELS APROPIATIVOS

Problema de la sección crítica:

N procesos con una sección crítica que no puede ser ejecutada por más de un proceso a la vez.

Necesidad de diseñar un protocolo de cooperación para garantizar exclusión mutua, progreso y espera limitada.

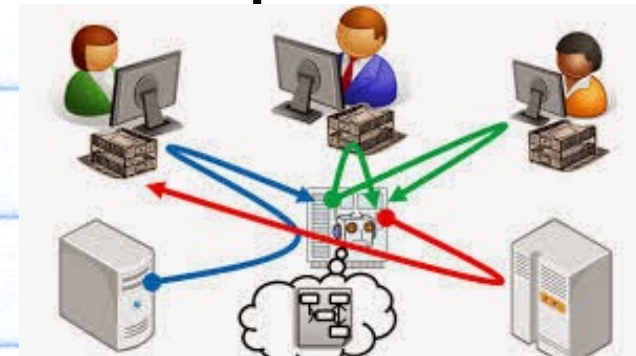
Kernels apropiativos vs. no apropiativos.

Problemas potenciales de condiciones de carrera en los kernels apropiativos.

Importancia de diseñar cuidadosamente los kernels apropiativos para evitar condiciones de carrera.

MECANISMOS DE SINCRONIZACIÓN PARA EVITAR CONDICIONES DE CARRERA EN SISTEMAS CONCURRENTES

El diseño de mecanismos de sincronización robustos es esencial para mantener la integridad y eficiencia de sistemas concurrentes, especialmente en entornos con kernels apropiativos.



DESAFÍOS EN LA IMPLEMENTACIÓN DE INSTRUCCIONES PARA CONTROL DE INTERRUPCIONES EN SISTEMAS CONCURRENTES

MUCHOS SISTEMAS UTILIZAN INSTRUCCIONES PARA DESACTIVAR PROBLEMAS DE INTERRUPCIONES (TESTANDSET() Y SWAP()).

LA IMPLEMENTACIÓN DE DICHAS INSTRUCCIONES NO ES TAREA SIMPL

EXISTEN DOS TIPOS DE SEMÁFOROS: **BINARIOS** Y **CONTADORES.**

LAS REGIONES CRÍTICAS SE PROTEGEN MEDIANTE CERROJOS EL CONTADOR PUEDE CAMBIAR DE VALOR SEGÚN UN DOMINIO NO RESTRINGIDO

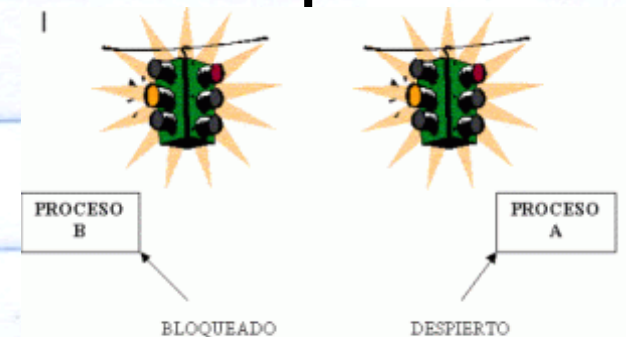
LOS BINARIOS SOLO PUEDEN TOMAR LOS VALORES DE 0 O 1

ESTOS SEMÁFOROS SE PUEDEN UTILIZAR PARA PRIORIZAR EL USO DE RECURSOS PARA PROCESOS CONCURRENTES

SINCRONIZACIÓN DE PROCESOS

LOS SEMÁFOROS SON HERRAMIENTAS MÁS SENCILLAS QUE LAS INSTRUCCIONES ES UNA VARIABLE S DE TIPO ENTERO QUE SOLO SE PUEDE ACCEDER POR MEDIO DE DOS FUNCIONES WAITO Y SIGNALO

Los semáforos actúan como controladores de acceso en entornos de múltiples procesos, permitiendo que solo un número limitado de procesos ingresen a una sección crítica. A diferencia de las instrucciones de bajo nivel



INCONVENIENTES DE LA

ESPERA ACTIVA:

La espera activa significa que un proceso en su sección crítica sigue ejecutando un bucle de comprobación en lugar de bloquearse, lo cual puede consumir ciclos de CPU innecesariamente y afectar el rendimiento en un

Implementación de la sistema multiprogramado.

LISTA DE PROCESOS EN ESPERA:

Se explica cómo se utiliza una lista de procesos en espera asociada a cada semáforo para gestionar los procesos bloqueados, facilitando su posterior desbloqueo.

AJUSTE DE LAS OPERACIONES

WAIT() Y SIGNAL():

Se sugiere modificar las operaciones wait() y signal() de los semáforos para eliminar la espera activa. En vez de usar bucles de espera, el proceso se bloqueará mediante operaciones específicas de bloqueo y desbloqueo.

FUNCIONAMIENTO DE WAIT() Y SIGNAL():

La operación wait() disminuye el valor del semáforo y, si este valor se vuelve negativo, bloquea el proceso. Por otro lado, la operación signal() aumenta el valor del semáforo y desbloquea un proceso en espera si es necesario.

SINCRONIZACIÓN DE PROCESOS

DEFINICIÓN DE INTERBLOQUEOS:

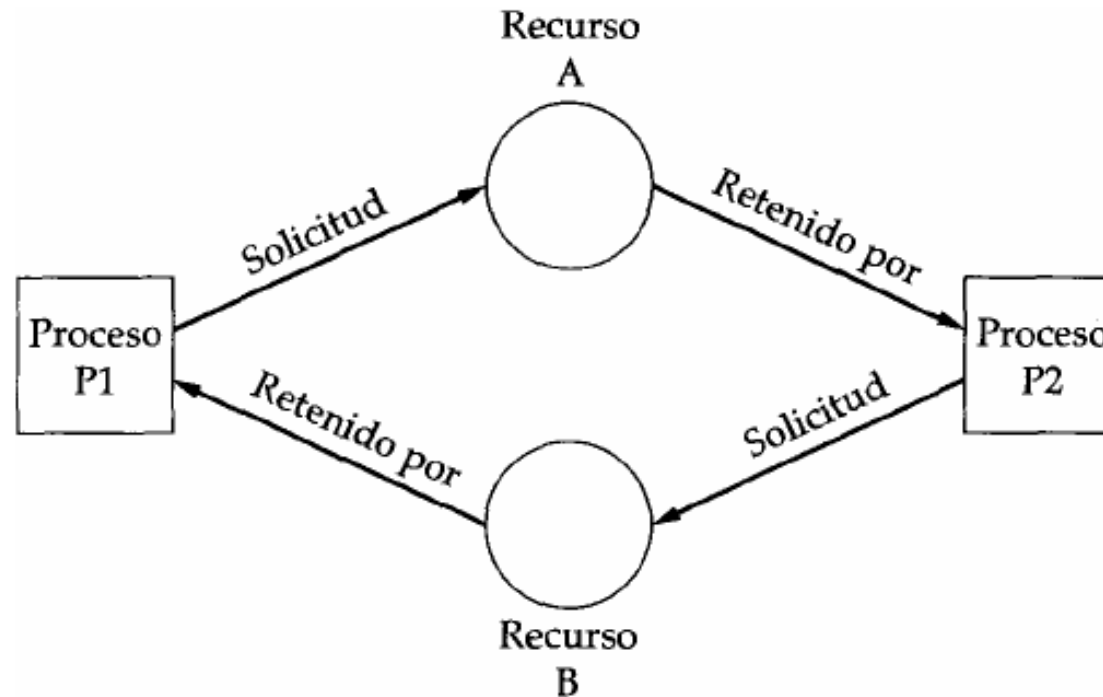
Los interbloqueos ocurren cuando dos o más procesos quedan esperando indefinidamente un evento que solo puede desencadenarse por las acciones de otro proceso que también está en espera.

ATOMICIDAD EN LA EJECUCIÓN DE OPERACIONES:

Es fundamental asegurar que las operaciones wait() y signal() se realicen de manera atómica para prevenir problemas de concurrencia.

EJEMPLO DE INTERBLOQUEO ENTRE PROCESOS:

El Proceso A bloquea el Recurso X y espera el Recurso Y, mientras que el Proceso B tiene bloqueado el Recurso Y y espera el Recurso X. Ambos procesos quedan indefinidamente esperando que el otro libere el recurso, generando un interbloqueo.



BLOQUEO INDEFINIDO O INANICIÓN:

Este concepto se refiere a una situación en la que algunos procesos quedan esperando indefinidamente en un semáforo, sin poder avanzar, lo que también se conoce como muerte por inanición.

CAUSAS Y CONSECUENCIAS DE INTERBLOQUEOS:

SE EXPLICAN LAS CAUSAS Y CONSECUENCIAS DE LOS INTERBLOQUEOS, ASÍ COMO SU IMPACTO EN LA EJECUCIÓN DEL SISTEMA.

INTRODUCCIÓN A LOS MONITORES COMO PRIMITIVA DE SINCRONIZACIÓN DE NIVEL MÁS ALTO:

SE INTRODUCE EL CONCEPTO DE MONITORES COMO UNA PRIMITIVA DE SINCRONIZACIÓN DE NIVEL MÁS ALTO QUE PROPORCIONA UNA INTERFAZ MÁS FÁCIL DE USAR PARA CONTROLAR EL ACCESO A RECURSOS COMPARTIDOS.

INTRODUCCIÓN AL PROBLEMA DEL BÚFER LIMITADO:

SE PRESENTA EL PROBLEMA DEL BÚFER LIMITADO COMO UN EJEMPLO CLÁSICO DE SINCRONIZACIÓN ENTRE PRODUCTORES Y CONSUMIDORES.

SOLUCIÓN UTILIZANDO SEMÁFOROS:

SE EXPLICA CÓMO SE PUEDE RESOLVER EL PROBLEMA DEL BÚFER LIMITADO UTILIZANDO SEMÁFOROS PARA CONTROLAR EL ACCESO AL BÚFER Y GARANTIZAR LA EXCLUSIÓN MUTUA.

SINCRONIZACIÓN DE PROCESOS

DESCRIPCIÓN DE SEMÁFOROS BINARIOS Y SU USO:

SE DESCRIBE EL CONCEPTO DE SEMÁFOROS BINARIOS Y SE EXPLICA CÓMO SE UTILIZAN PARA SINCRONIZAR PROCESOS Y GARANTIZAR LA EXCLUSIÓN MUTUA